Première manipulation de Neo4j

Créer un graphe et réaliser les opérations CRUD — le restaurant

Objectifs

- Connexion à Neo4j
- Construire un petit graphe pas à pas,
- lire, modifier et supprimer un graphe avec Cypher (CRUD : Create, Read, Update, Delete).

Prérequis

Neo4j est installé, soit avec Docker (option 1), soit avec Neo4j Desktop (option 2). Seule l'étape 1 dépend de votre option ; toutes les étapes suivantes sont identiques pour tout le monde.

Étape 1 — Se connecter à Neo4j

Option 1 : Docker

1. Lancer Docker Desktop et attendre que le moteur soit démarré.
2. Vérifier que le conteneur tourne :

docker ps

Si le conteneur `neo4j` n'apparaît pas, le redémarrer (sans refaire de `docker run`) :

docker start neo4j

3. Ouvrir **http://localhost:7474** dans un navigateur.
4. Choisir **No authentication**, puis cliquer sur **Connect**.

Option 2 : Neo4j Desktop

1. Lancer Neo4j Desktop.
2. Démarrer votre instance (**Start**) et attendre qu'elle soit indiquée comme active.
3. Ouvrir l'outil de requête de Desktop, ou **http://localhost:7474** dans un navigateur.
4. Se connecter avec l'utilisateur `neo4j` et le mot de passe choisi à la création de l'instance.

Étape 2 — CREATE : construire le graphe

Un restaurant veut enregistrer ses clients, ses plats et leurs ingrédients, savoir qui a parrainé qui, et garder les notes données par les clients aux plats.

Voici le graphe que vous allez construire :

Élément	Contenu
Clients	Sara (Casablanca), Youssef (Fès), Lina (Casablanca)
Plats	Tajine de poulet (85 DH, plat), Couscous royal (95 DH, plat), Salade marocaine (35 DH, entrée)
Ingrédients	Poulet, Semoule (allergène), Tomate

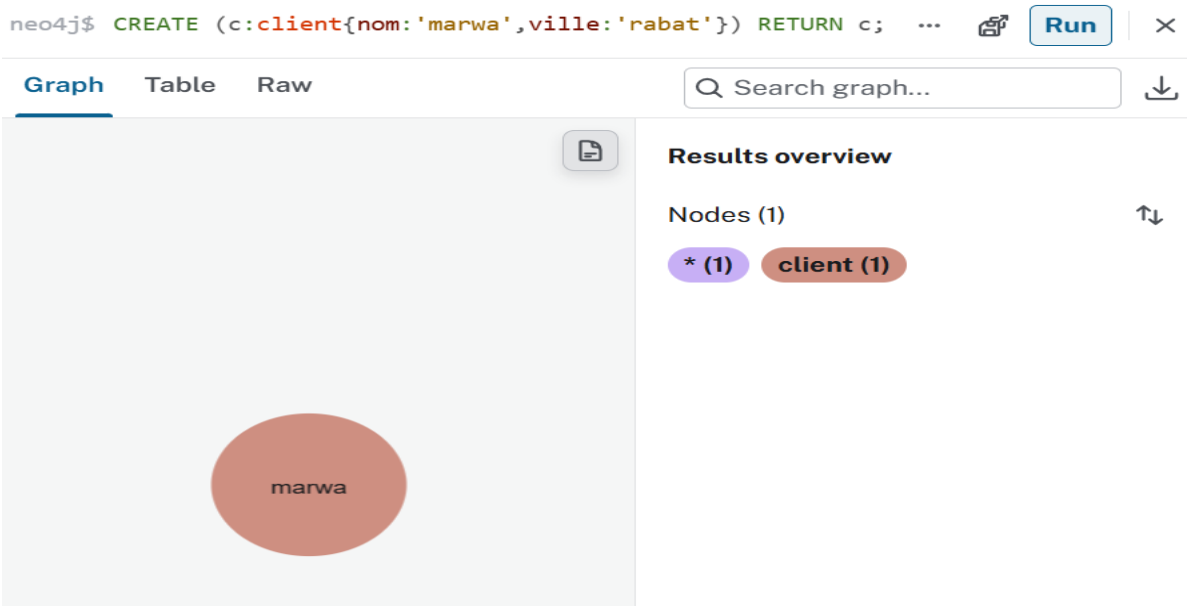
Élément	Contenu
PARRAINE	Sara parraine Youssef et Lina
COMPOSE_DE	Tajine : Poulet ; Couscous : Semoule, Poulet ; Salade : Tomate
A_NOTE {note}	Sara : Tajine (5) ; Youssef : Couscous (4) ; Lina : Tajine (4) ; Lina : Salade (3)

3.1 Créer un premier nœud

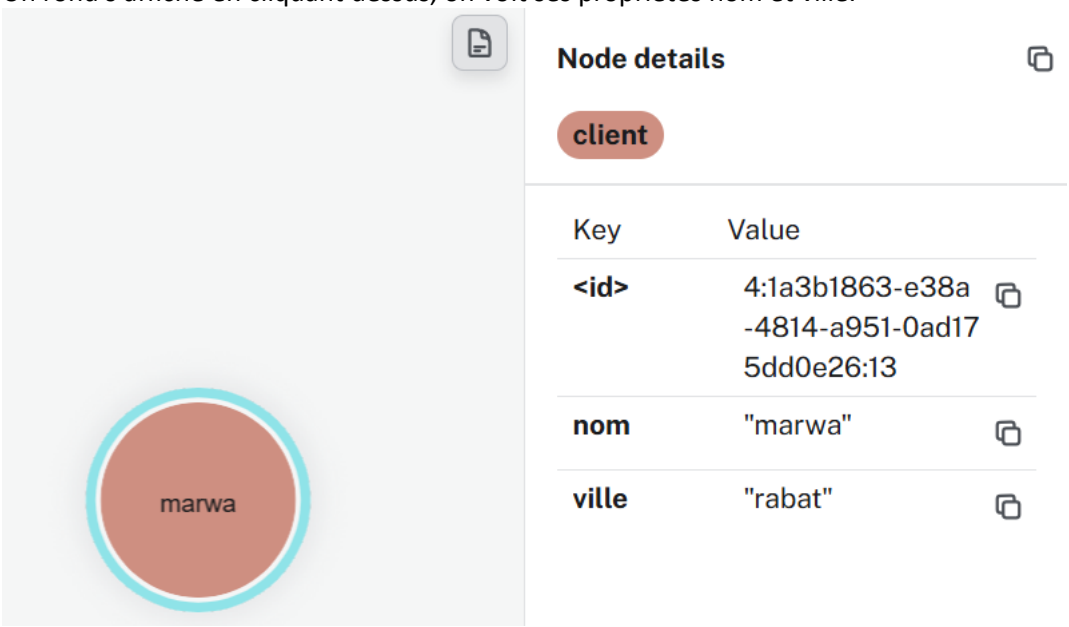
Nous allons créer le premier **noeud** qui sera inserer en rajoutant un **label** "Client" et ses **propriétés**:{nom: ..., ville : .....} en utilisant la commande suivante :

```
neo4j$ CREATE (client{nom:'marwa',ville:'rabat'})
```

On peut aussi utiliser la syntaxe suivante qui permet d'afficher l'éléments enregistrer en utilisant une variable c dans RETURN



Un rond s'affiche en cliquant dessus, on voit ses propriétés nom et ville.



3.2 Créer les autres clients, les plats et les ingédients

```
$
```

```
1  CREATE
2    (:Client {nom: 'Sara', ville: 'Casablanca'}),
3    (:Client {nom: 'Youssef', ville: 'Fès'}),
4    (:Client {nom: 'Lina', ville: 'Casablanca'}),
5
6    (:Plat {nom: 'Tajine de poulet', prix: 85, categorie:
   'plat'}),
7    (:Plat {nom: 'Couscous royal', prix: 95, categorie:
   'plat'}),
8    (:Plat {nom: 'Salade marocaine', prix: 35, categorie:
   'entrée'}),
9
10   (:Ingredient {nom: 'Poulet', allergene: false}),
11   (:Ingredient {nom: 'Semoule', allergene: true}),
12   (:Ingredient {nom: 'Tomate', allergene: false});
```

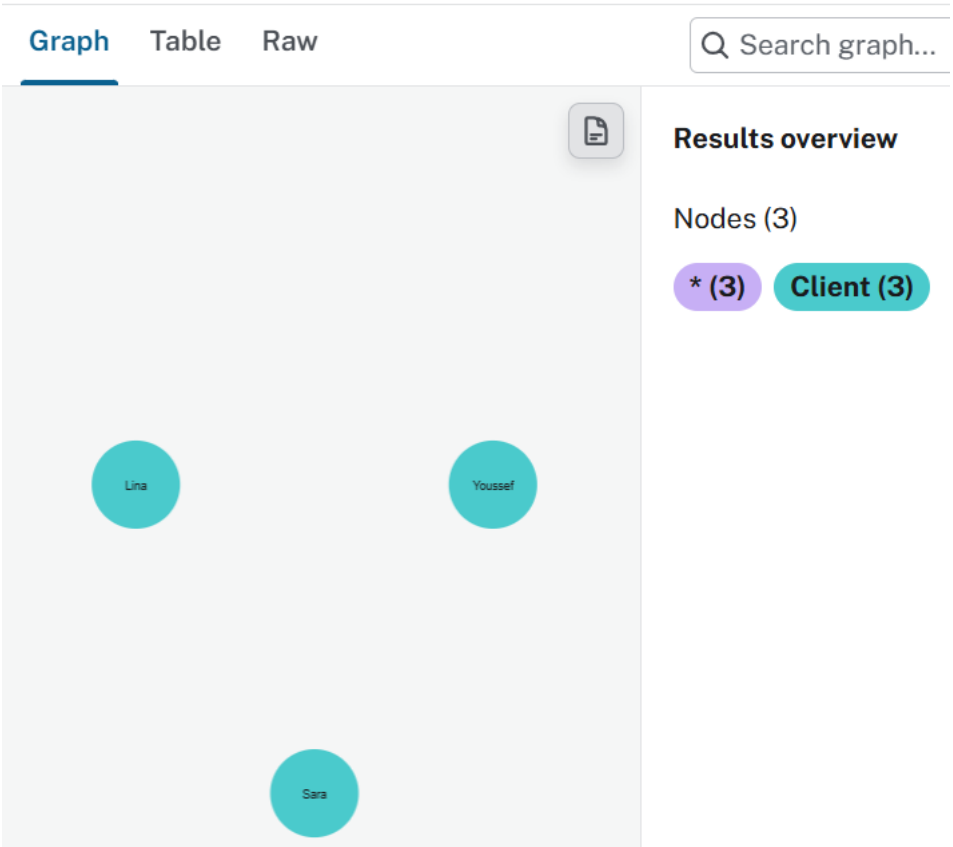
Run

Created 9 nodes, set 21 properties, added 9 labels

Completed after 2 769 m

Afficher par la suite les données avec la requête suivante:

```
neo4j$ MATCH(c:Client) RETURN c;
```



On peut aussi vérifier le nombre de noeud créé :

```
neo4j$ MATCH(n) RETURN count(n);
```

Run

Table Raw Search table... Graph chip

count(n)
14

3.4 Créer les relations

Pour relier deux nœuds qui existent déjà, on les **retrouve** d'abord avec `MATCH`, puis on crée la relation entre eux :

```
MATCH (s:Client {nom: 'Sara'}), (y:Client {nom: 'Youssef'})
CREATE (s)-[:PARRAINE]->(y);
```

À faire : créer de la même façon la relation **Sara PARRAINE Lina**.

La composition des plats :

```
MATCH (t:Plat {nom: 'Tajine de poulet'}),
      (co:Plat {nom: 'Couscous royal'}),
      (sa:Plat {nom: 'Salade marocaine'}),
      (po:Ingredient {nom: 'Poulet'}),
      (se:Ingredient {nom: 'Semoule'}),
      (to:Ingredient {nom: 'Tomate'})

CREATE (t)-[:COMPOSE_DE]->(po),
       (co)-[:COMPOSE_DE]->(se),
       (co)-[:COMPOSE_DE]->(po),
       (sa)-[:COMPOSE_DE]->(to);
```

Les notes : une relation peut, elle aussi, avoir des propriétés.

Saved Run

```
MATCH (s:Client {nom: 'Sara'}),
      (t:Plat {nom: 'Tajine de poulet'})
CREATE (s)-[:A_NOTE {note: 5}]->(t);

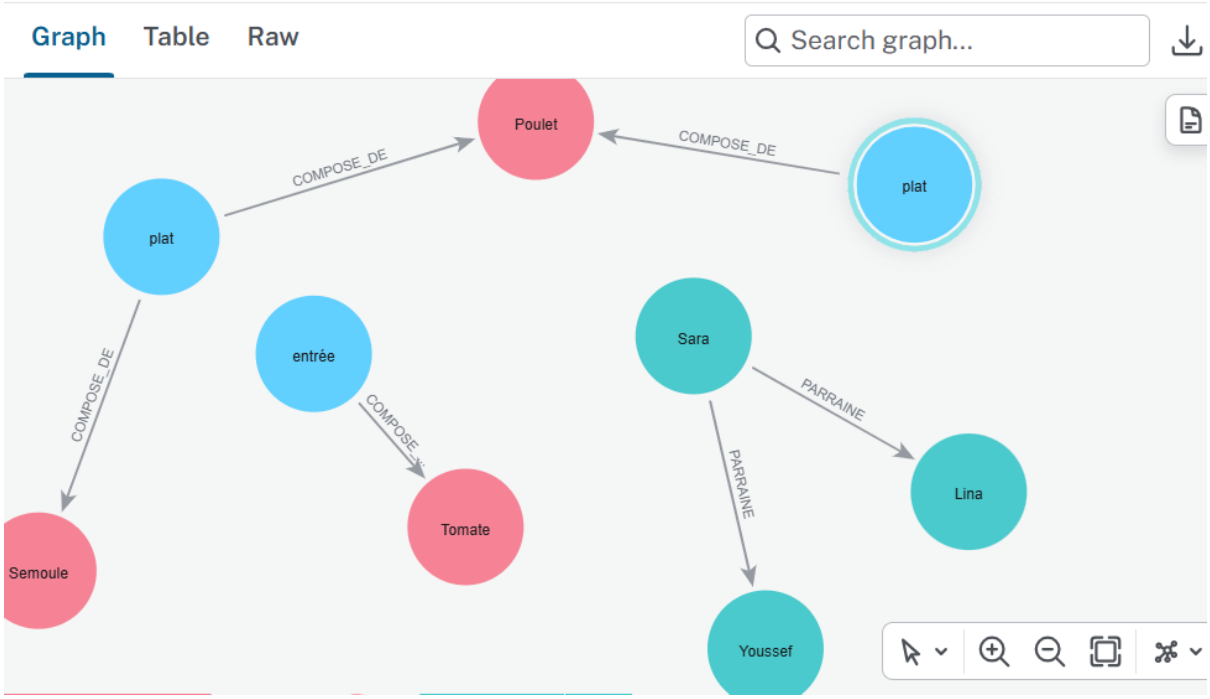
MATCH (y:Client {nom: 'Youssef'}), (co:Plat {nom: 'Couscous royal'})
CREATE (y)-[:A_NOTE {note: 4}]->(co);

MATCH (l:Client {nom: 'Lina'}), (t:Plat {nom: 'Tajine de poulet'}), (sa:Plat {nom:
'Salade marocaine'})
CREATE (l)-[:A_NOTE {note: 4}]->(t), (l)-[:A_NOTE {note: 3}]->(sa);
```

3.5 Visualiser le graphe complet

```
$ MATCH (a)-[r]->(b) RETURN a, r, b;
```

Saved ··· Run



Étape 3 — READ : interroger le graphe

Une requête de lecture suit toujours le même schéma : `MATCH` (le motif cherché) → `WHERE` (les conditions) → `RETURN` (ce qu'on affiche).

3.1 Afficher le nom et la ville de tous les clients.

```
1 MATCH (c:Client)
2 RETURN c.nom, c.ville;
```

Saved ··· Run

Table Raw | Search table... | Graph chip

c.nom	c.ville
"Sara"	"Casablanca"
"Youssef"	"Fès"
"Lina"	"Casablanca"

3.2 Afficher les clients qui habitent à Casablanca.

```
1 MATCH (c:Client {ville: 'Casablanca'})
2 RETURN c.nom;
```

Saved ··· Run

Table Raw Search table... Graph chip

c.nom
1 "Sara"
2 "Lina"

3.3 Afficher les plats qui coûtent plus de 50 DH.

```
1 MATCH (p:Plat)
2 WHERE p.prix > 50
3 RETURN p.nom, p.prix;
```

Table Raw Search table...

p.nom	p.prix
1 "Tajine de poulet"	85
2 "Couscous royal"	95

3.4 Afficher la carte, du plat le moins cher au plus cher.

```
1 MATCH (p:Plat)
2 RETURN p.nom, p.prix
3 ORDER BY p.prix;
```

Table Raw Search table...

p.nom	p.prix
1 "Salade marocaine"	35
2 "Tajine de poulet"	85
3 "Couscous royal"	95

3.5 Quels clients Sara a-t-elle parrainés ?

```
MATCH (:Client {nom: 'Sara'})-[:PARRAINE]->(f:Client)
RETURN f.nom;
```

Table Raw Search table...

f.nom
"Youssef"
"Lina"

Étape 4 — UPDATE : modifier les données

On modifie en deux temps : `MATCH` pour trouver l'élément, puis `SET` (ajouter ou modifier) ou `REMOVE` (retirer).

4.1 Youssef donne son adresse email.

```
MATCH (c:Client {nom: 'Youssef'})
SET c.email = 'youssef@exemple.ma'
RETURN c;
```

Graph Table Raw



4.2 Youssef demande la suppression de son email.

```
MATCH (c:Client {nom: 'Youssef'})
REMOVE c.email
RETURN c;
```

Graph Table Raw



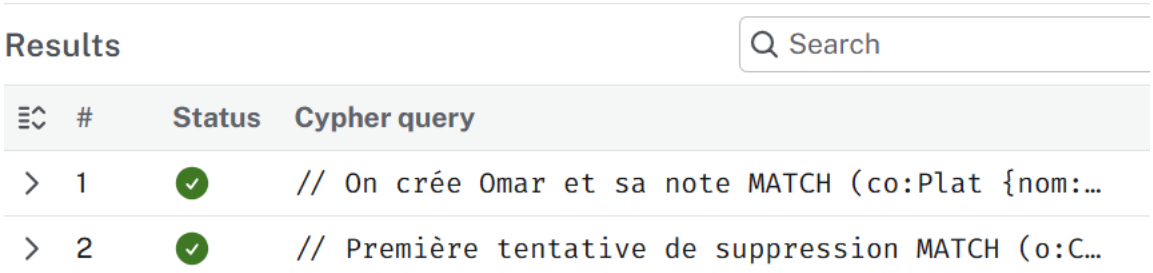
Étape 5 — DELETE : supprimer des données

5.1 Un client de passage, Omar, note le couscous puis demande la suppression de son compte.

```
// On crée Omar et sa note
MATCH (co:Plat {nom: 'Couscous royal'})

CREATE (:Client {nom: 'Omar', ville: 'Tanger'})-[:A_NOTE {note: 2}]->(co);

// Première tentative de suppression (sans DETACH )
MATCH (o:Client {nom: 'Omar'})
DETACH DELETE o;
```



DELETE refuse de supprimer un nœud relié, pour ne pas laisser une relation sans extrémité. Il faut DETACH DELETE, qui supprime d'abord ses relations, puis le nœud.

5.2 Lina retire sa note de la salade (la relation disparaît, les nœuds restent).

```
MATCH (:Client {nom: 'Lina'})-[n:A_NOTE]->(:Plat {nom: 'Salade
marocaine'})
DELETE n;
```